SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1 **Total Marks:100**

Annexure A

Assignment

Code of Conduct:

I E. G. Abhinaya (192320051) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

e.g. abhinaya

Signature of the student:

Course Code & Name

CSA11 – Object Oriented Analysis and Design

Assignment Title	Object-Oriented Analysis and UML-Based Design of an Intelligent Food Waste Reduction and Redistribution System (IFWRRS)
Course Outcome(s) – CO	CO1: Analyse Object-Oriented Systems Development concepts and methodologies to understand and model a real-world software problem. CO2: Apply UML modelling techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and relationship identification.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 2 – Zero Hunger SDG 11 – Sustainable Cities and Communities SDG 12 – Responsible Consumption and Production SDG 13 – Climate Action

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an Intelligent Food Waste Reduction and Redistribution System (IFWRRS) that connects restaurants, hotels, supermarkets, food suppliers, charitable organizations, delivery partners, and administrators to reduce avoidable food waste and facilitate timely redistribution of surplus food.

The system shall allow registered food providers to record surplus food, specify quantity, type, preparation time, expiry period, storage requirements, and pickup location. The system shall identify suitable recipient organizations based on food requirements, location, quantity, urgency, and availability.

Recipient organizations shall be able to view available food donations, request suitable items, confirm collection, and update distribution status. Delivery partners shall receive pickup and delivery assignments and update the transportation status. Administrators shall monitor food donations, organizations, deliveries, users, and system reports.

The system shall generate notifications when surplus food is approaching its expiry period and support intelligent matching between food providers and recipient organizations. Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams.

Students shall analyse object responsibilities and apply suitable design principles and design patterns to achieve high cohesion, low coupling, reusability, maintainability, and scalability.

Constraints

The design shall clearly distinguish different stakeholders and their responsibilities. All identified classes, attributes, operations, and relationships shall be traceable to the identified use cases.

The system shall consider food expiry, notification timing, matching criteria, delivery status, user roles, data consistency, and transaction management. UML diagrams shall be mutually consistent and shall accurately represent the proposed system behaviour.

The final submission shall include assumptions, constraints, object responsibility analysis, design-pattern justification, and traceability between requirements, use cases, and design elements.

Tools Can Be Used

- StarUML
- Visual Paradigm
- Draw.io
- PlantUML
- Papyrus Tools
- ArgoUML
- Enterprise Architect

- Lucidchart

ANSWER

INTRODUCTION

The Intelligent Food Waste Reduction and Redistribution System (IFWRRS) is a software application that is designed to reduce wastage of food that would otherwise be thrown away. It facilitates the linkages between entities that have surplus food and non-profit organizations that need to distribute the food. Food providers such as restaurants, hotels, supermarkets, and suppliers can be registered and record the surplus that they have. They also provide the details of the type of food, the quantity, the preparation period, the expiry date, the storage needs, and the pickup location. In addition, the application matches the surplus food with the recipient organizations based on several indicators such as the needs of the recipient, the location of the entities, the quantities required and available, the urgency, and the current availability. It aims to help redistribute surplus food to the right recipients before the expiry date. IFWRRS facilitates the delivery logistics to ensure that the food is picked up and delivered to the recipient organizations. The administrators manage the application including the users, food donations, organizations, delivery activities, and reports. The system can also send out alerts to the organizations about the expiry dates of their donated items to help them prioritize the items that need to be claimed first. In designing the application, an Object-Oriented Analysis and Design (OOAD) approach is useful to identify the actors, objects, classes, attributes, methods, and relationships. Once the models have been developed, it will be possible to construct various Unified Modeling Language (UML) views including Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment diagrams.

OBJECT-ORIENTED ANALYSIS OF THE IFWRRS ESSAY

The Object-Oriented Analysis of the Intelligent Food Waste Reduction and Redistribution System focuses on identifying the main users, objects, and classes as well as their responsibilities.

Main Actors

• Food Provider – records and manages surplus food.

• Recipient Organization – searches and requests appropriate donations.

• Delivery Partner – picks up and delivers food donations.

• Administrator – oversees all operations including users, donations, deliveries, and reports.

Main Classes

• User – stores shared information and handles authentication procedures.

• Food Donation – stores information on food type, quantity, preparation time, expiry date, storage requirements and pickup location.

• Recipient Organization – stores food requirements and recipient donation information.

• Delivery – oversees food pickup, transportation, and delivery.

• Matching Engine – matches donations based on location, quantity, food requirements, urgency and availability.

• Notification – sends out reminders, expiry warnings, and request confirmations.

A Food Provider creates several Food Donations. The Matching Engine cross-references the available donations with the needs of Recipient Organizations. A Delivery can be organized by a Delivery Partner for a given Food Donation. The Notification system alerts all relevant users concerning impending expiry, requests, or deliveries.

Objective of the Analysis

The objective of this analysis is to define the system's objects and their responsibilities ensuring high cohesion, low coupling, reusability, and system maintainability and scalability.

Identification of Classes

The following classes were identified from the use case descriptions:

User

The User class stores shared information on all users and handles authentication and login procedures.

Attributes:

• userId

• name

• email

• phone

• password

• status

Methods:

• login()

• logout()

• updateProfile()

The User class serves as the parent class for the FoodProvider, RecipientOrganization, DeliveryPartner, and Administrator classes, which means that it defines default attributes and methods that can be overridden by the child classes.

The important classes identified from the scenario are as listed below:

User

The User class will contain common information about users.

Attributes: userId name email phone password status

Methods: login() logout() updateProfile()

The classes FoodProvider, RecipientOrganization, DeliveryPartner, and Administrator can be made to inherit from this class.

1.FoodProvider

The attributes for this class are: providerId organizationName, providerType address

Methods: recordDonation() updateDonation() cancelDonation() viewDonationStatus()

RecipientOrganization

The attributes for this class are: organizationId organizationName organizationType location

Methods: viewDonations() searchFood() requestDonation() confirmCollection() updateDistributionStatus()

2.DeliveryPartner

The attributes for this class are: partnerId vehicleType availabilityStatus currentLocation

Methods: viewAssignment() acceptAssignment() pickupFood() updateTransportStatus() confirmDelivery()

FoodDonation

This is a very important class.

Attributes: donationId foodType quantity preparationTime expiryTime storageRequirement pickupLocation donationStatus

Methods: createDonation() updateDonation() calculateUrgency() changeStatus()

3.FoodRequirement

This class contains information about the requirements of the recipient organizations.

Attributes: requirementId foodType requiredQuantity storageRequirement urgency

Methods: checkCompatibility() updateRequirement()

MatchingEngine

The Matching Engine will contain methods that will perform matching operations when searching for a suitable food donation to fulfill a food requirement.

Attributes: matchingCriteria matchingScore

Methods: findMatches() calculateMatchScore() rankRecipients()

4.Delivery

Deliveries will be represented by this class.

Attributes: deliveryId pickupLocation destination assignedTime deliveryStatus

Methods: assignPartner() confirmPickup() updateStatus() confirmDelivery()

Notification

This class will send out messages or notifications to users.

Attributes: notificationId message notificationType timestamp status

Methods: send() markAsRead()

NON-FUNCTIONAL REQUIREMENTS

Performance

The system should be able to locate potential recipients of donated food items quickly before the expiration date.

Security

User's credentials and the organization's data should be safe in the system.

Scalability

The system should be able to handle a high load of providers, recipients, donations, and delivery services that are expected to be in the network.

Reliability

The information about donations and delivery should always be updated to guarantee that redistribution will happen smoothly.

Maintainability

It should be possible to update separate classes and interfaces without disrupting the rest of the system.

Usability

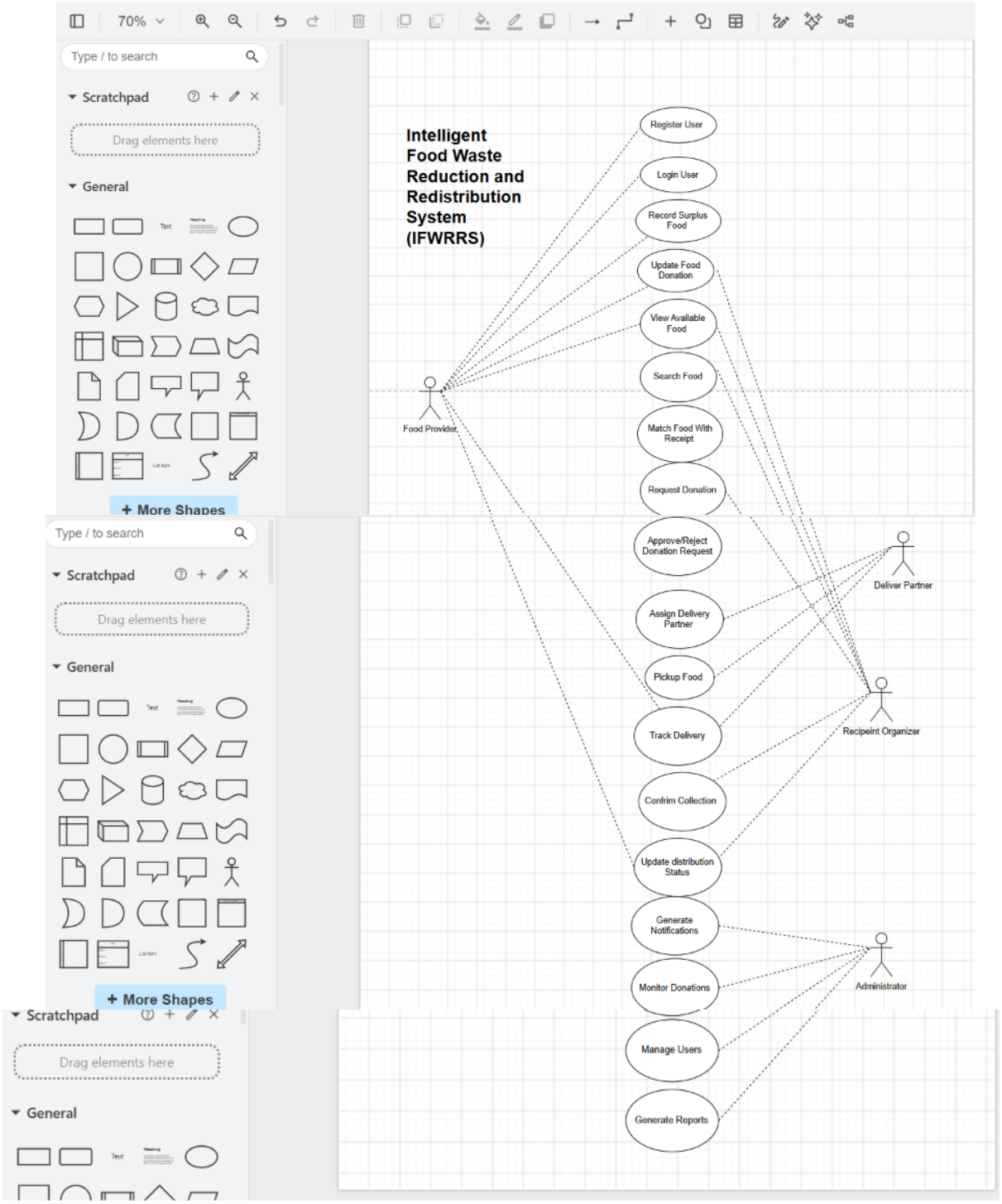
The system should be user friendly enabling users to report donations, request, and track delivery of the food items effectively and efficiently.

RESULTS

The Object-Oriented Analysis of the Intelligent Food Waste Reduction and Redistribution System (IFWRRS) identified the main actors, objects, classes, attributes, methods, and relationships needed for the system.The analysis identified Food Provider, Recipient Organization, Delivery Partner, and Administrator as the main actors.Important classes such as User, FoodDonation, FoodRequirement, MatchingEngine, Delivery, and Notification were defined along with their respective roles.The model allows for tracking surplus food, matching donations with appropriate recipients, organizing deliveries, monitoring distribution, and sending notifications to all stakeholders. The findings support the development of the application that will facilitate the redistribution of surplus food and ensure that it meets the needs of the recipients and is delivered in good time. The UML diagrams will aid in the implementation process since they show how the application will come about, including the interactions and activities that will take place. This will support the design and construction of an effective and efficient application that will reduce food wastage and support sustainability. expiry alerts.The analysis also lays the groundwork for creating UML diagrams and applying object-oriented principles like high cohesion, low

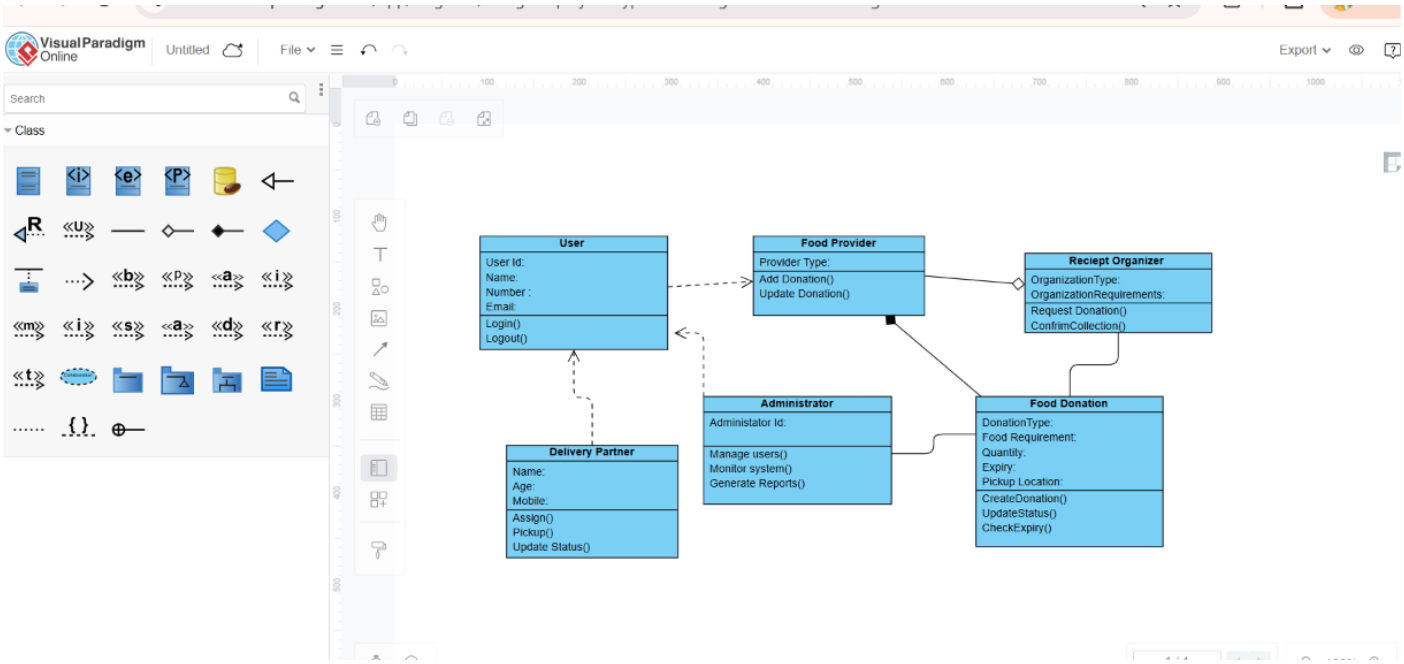
coupling, encapsulation, reusability, maintainability, and scalability.In summary, the proposed model offers a structured way to improve food redistribution and reduce avoidable food waste.

USECASE DIADRAM:



LINK: https://app.diagrams.net/

CLASS DIAGRAM:



LINK:https://online.visualparadigm.com/app/diagrams/#diagram:proj=0&type=ClassDiagram&width=11&height=8.5&unit=inch

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding, Object Identification & Requirements Analysis (CO1)	**10**	Clearly analyses the problem domain and accurately identifies all major actors, objects, system responsibilities, and requirements from the given scenario.	Identifies most actors, objects, and requirements with minor gaps or inaccuracies.	Identifies basic actors and objects but analysis is incomplete or contains some inconsistencies.	Poor understanding of the problem; major actors, objects, and requirements are missing or incorrectly identified.
Use Case Modelling & Actor Identification (CO2)	**10**	Correctly identifies all relevant actors and use cases; Use Case Diagram clearly represents system boundary, relationships, and interactions.	Most actors and use cases are correctly identified with minor modelling errors.	Basic actors and use cases are identified, but relationships or system boundary need improvement.	Actors and use cases are poorly identified; diagram is incomplete or inconsistent.
Class Identification, Attributes, Methods & Relationships (CO2/CO3)	**15**	Classes, attributes, methods, associations, aggregation/composition, inheritance, and dependencies are accurately identified and logically justified.	Class structure is mostly correct with minor errors in attributes, methods, or relationships.	Basic classes and relationships are identified but lack detail or contain several modelling issues.	Classes, attributes, methods, or relationships are missing, inappropriate, or poorly structured.
UML Behavioural & Interaction Modelling (CO2)	**15**	Sequence, Activity, and State Transition Diagrams accurately represent system behaviour and are fully consistent with the identified use cases and classes.	Behavioural diagrams are mostly correct with minor inconsistencies.	Basic behavioural diagrams are provided but some interactions, activities, or states are incomplete.	Behavioural diagrams are missing, unclear, or inconsistent with the proposed system.
UML Architectural	**15**	Package, Component, and Deployment Diagrams	Architectural diagrams are	Basic architectural	Architectural diagrams are

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Modelling & Design Evaluation (CO3)		clearly represent modular structure, system components, dependencies, and deployment environment with strong justification.	mostly correct with minor omissions or modelling errors.	representation is provided but lacks clarity or sufficient justification.	incomplete, inappropriate, or inconsistent with the system design.
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	**15**	Responsibilities are appropriately assigned; design axioms, cohesion, coupling, encapsulation, abstraction, and suitable design patterns are clearly justified and effectively applied.	Design principles and patterns are appropriately applied with minor limitations in justification.	Some design principles are identified, but application or justification is limited.	Design principles, axioms, responsibilities, or patterns are absent or incorrectly applied.
Consistency, Traceability & Overall Design Quality (CO3)	**10**	All UML models are mutually consistent, requirements are traceable to use cases/classes, and the overall design is complete, feasible, scalable, and maintainable.	Design is largely consistent and traceable with minor gaps.	Some consistency and traceability are demonstrated, but important links are missing.	Major inconsistencies exist between requirements, use cases, classes, and UML diagrams.
Reflection, SDG Relevance & Learning Outcomes	**10**	Provides insightful reflection on design decisions, challenges, OOAD concepts learned, and clearly establishes the relevance of the selected SDGs.	Provides good reflection with reasonable connection to design decisions and SDGs.	Reflection is present but generic with limited discussion of learning and SDG relevance.	Reflection is missing, superficial, or unrelated to the assignment and SDGs.
Total	**100**				

Deliverables

To receive full credit, each submission must include the following:

1. **Problem Analysis and Requirement Summary**
2. **Actor Identification**
3. **Use Case Descriptions**
4. **Use Case Diagram**
5. **Object and Class Identification**
6. **Class Diagram**
7. **Sequence/Interaction Diagram(s)**
8. **Activity Diagram**
9. **State Transition Diagram**
10. **Package Diagram**
11. **Component Diagram**
12. **Deployment Diagram**
13. **Object Responsibility Analysis**
14. **Design Axioms/Principles and Design Pattern Justification**
15. **Assumptions and Constraints**
16. **Requirement-to-Use Case/Class Traceability**
17. **Implementation/Prototype and Results**
18. **Reflection on Design Decisions and Learning Outcomes**
19. **GitHub Upload**